EJERCICIO. MODULO 02. CLASES Y OBJETOS

Práctica de OOP en C#: clase, objeto, propiedades, métodos y encapsulación

El objetivo de este ejercicio es seguir programando con .NET y C# y aprender a usar CLASES Y OBJETOS

Instrucciones: crea un proyecto de Consola y resuelve cada ejercicio. Las soluciones propuestas están debajo de cada enunciado para autocorregirte.

Ejercicios y soluciones

1) Crear una clase 'Libro'

Crea una clase Libro con 2 propiedades públicas: Titulo (string) y Autor (string). En Main crea 2 libros y muéstralos por consola.

Solución propuesta:

```
public class Libro
{
    public string Titulo { get; set; }
    public string Autor { get; set; }
}

// En Main:
var l1 = new Libro { Titulo = "El Quijote", Autor = "Cervantes" };
var l2 = new Libro { Titulo = "1984", Autor = "George Orwell" };

Console.WriteLine($"{l1.Titulo} - {l1.Autor}");
Console.WriteLine($"{l2.Titulo} - {l2.Autor}");
```

2) Método 'Mostrar' en 'Libro'

Añade a Libro un método Mostrar() que devuelva un string: "Titulo (Autor)". Imprime la descripción.

Solución propuesta:

```
public class Libro
{
    // PROPIEDADES
    public string Titulo { get; set; }
    public string Autor { get; set; }
      // METODO UNICO
    public string Mostrar() => Console.WriteLine($"{Titulo} ({Autor})");
}

// En Main:
var libro = new Libro { Titulo = "1984", Autor = "George Orwell" };
Libro.Mostrar();
```

3) Clase 'Persona' con método Saludar

Crea Persona con Nombre y Edad. Crea Saludar() que imprima "Hola, soy X".

Solución propuesta:

```
public class Persona
{
    public string Nombre { get; set; }
    public int Edad { get; set; }

    public void Saludar()
    {
        Console.WriteLine($"Hola, soy {Nombre}");
    }
}

// En Main:
var p = new Persona { Nombre = "Ana", Edad = 30 };
p.Saludar();
```

4) Constructor obligatorio en 'Tarea'

Crea Tarea con Titulo y Completada (si/no).

Constructor que reciba titulo y no permita vacío (lanza excepción).

Solución propuesta:

```
public class Tarea
{
    public string Titulo { get; private set; }
    public bool Completada { get; private set; }

    public Tarea(string titulo)
    {
        if (string.IsNullOrWhiteSpace(titulo))
            throw new ArgumentException("El título es obligatorio.");
        Titulo = titulo.Trim();
        Completada = false;
    }
}
```

5) Método MarcarCompleta()

En Tarea, crea MarcarCompleta() que ponga Completada a true y pruébalo en Main.

Solución propuesta:

```
public void MarcarCompleta() => Completada = true;

// En Main:
var t = new Tarea("Estudiar C#");
t.MarcarCompleta();
Console.WriteLine(t.Completada); // True
```

6) Encapsulación: CuentaBancaria sin set público

Crea CuentaBancaria con Saldo (decimal) sin set público. Método Ingresar(importe) que sume. Si importe <= 0, lanza excepción.

Solución propuesta:

```
public class CuentaBancaria
{
    public decimal Saldo { get; private set; }
```

```
    public void Ingresar(decimal importe)
    {
        if (importe <= 0) throw new ArgumentException("El importe debe ser positivo.");
        Saldo += importe;
    }
}
```

7) Regla extra: Retirar() con saldo insuficiente

En CuentaBancaria, añade Retirar(importe). No permitir retirar más del saldo.

Solución propuesta:

```
public void Retirar(decimal importe)
{
    if (importe <= 0) throw new ArgumentException("El importe debe ser positivo.");
    if (importe > Saldo) throw new InvalidOperationException("Saldo insuficiente.");
    Saldo -= importe;
}
```

8) Propiedad con validación (setter)

Crea Producto con propiedad Nombre que no permita null/vacío y haga Trim().

Solución propuesta:

```
public class Producto
{
    private string _nombre;

    public string Nombre
    {
        get => _nombre;
        set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Nombre obligatorio");
            _nombre = value.Trim();
        }
    }
}
```

9) Lista de objetos (List<T>)

Crea 3 tareas en una List<Tarea> y recórrela con foreach para imprimir Titulo y Completada.

Solución propuesta:

```
var tareas = new List<Tarea>
{
    new Tarea("Comprar pan"),
    new Tarea("Estudiar"),
    new Tarea("Hacer ejercicio")
};

foreach (var t in tareas)
{
    Console.WriteLine($"{t.Titulo} - Completada: {t.Completada}");
}
```

10) Método Renombrar con validación

En Tarea, crea Renombrar(nuevoTitulo) que no permita vacío y haga Trim(). Pruébalo.

Solución propuesta:

```
public void Renombrar(string nuevoTitulo)
{
    if (string.IsNullOrWhiteSpace(nuevoTitulo))
        throw new ArgumentException("El nuevo título no puede estar vacío.");
    Titulo = nuevoTitulo.Trim();
}

// En Main:
var t = new Tarea("  Comprar  ");
t.Renombrar("  Comprar leche  ");
Console.WriteLine(t.Titulo); // "Comprar leche"
```